



INSTITUTO POLITÉCNICO NACIONAL

UNIDAD PROFESIONAL INTERDISCIPLINARIA DE INGENIERÍA Y CIENCIAS SOCIALES Y ADMINISTRATIVAS

UPIICSA

LICENCIATURA EN CIENCIAS DE LA INFORMÁTICA

DOCUMENTACIÓN DE PRUEBAS DE SOFTWARE

Proyecto: Calculadora V2.0

Asignatura

Modelos de Prueba de Software

Equipo

Santiago Longino Alex Oliveratoom — Analista

Gonzales Bravo Zahira Yexalen — Diseñadora

Tlecuitl Hernández Fernando Raul — Desarrollador

Rosas Rosas Raul — Tester

Secuencia

5CV50

Índice

Índice	2
1. Introducción.....	4
2. Objetivos.....	4
2.1 Objetivo general	4
2.2 Objetivos específicos	4
3. Alcance.....	5
4. Descripción del proyecto	5
5. Estrategia de pruebas	6
5.1 Tipos de pruebas.....	6
5.2 Verificación	6
5.3 Validación	7
5.4 Estrategia general	7
6. Planeación	7
6.1 Cronograma	7
6.2 Roles y responsabilidades.....	8
7. Configuración del entorno de pruebas	8
Hardware	8
Software	8
8. Pruebas unitarias	9
9. Pruebas de integración	11
9.1 Modelo descendente.....	11
10. Cobertura de código	12
11. Especificaciones de prueba.....	13
12. Casos de prueba	13
13. Pruebas de sistema.....	15
13.1 Caja negra	15
13.2 Funcionales	15
13.3 Usabilidad	16
13.4 Rendimiento	16
14. Pruebas de regresión	16
14.1 Diferencias identificadas entre V1 y V2.0	16
14.2 Casos de regresión sobre funciones existentes	17
15. Matriz de rastreo de requisitos y pruebas.....	17
16. Registro de defectos	18
17. Informe de Pruebas Final.....	19
17.1 Actividades realizadas.....	19

17.2 Cobertura	20
17.3 Resultados	20
17.4 Conclusiones	20
18. Conclusiones generales	21
19. Referencias	21
20. Anexos	22
Anexo A. Funciones identificadas en el código de la Calculadora V2.0	22
Anexo B. Requerimientos originales (documento de análisis)	22
Anexo C. Bocetos de referencia del documento de diseño	22

1. Introducción

La Calculadora V2.0 es una aplicación de escritorio desarrollada en Python con la biblioteca Tkinter por el equipo del proyecto “Calculadora” de la asignatura Modelos de Prueba de Software (secuencia 5CV50, UPIICSA-IPN). El sistema permite realizar las cuatro operaciones aritméticas básicas, muestra los resultados con dos decimales, conserva un historial de operaciones durante la sesión, permite reutilizar el último resultado obtenido y valida dos condiciones específicas: la división entre cero y el uso de un solo punto decimal por operando.

Contar con un programa que funcione en apariencia no es suficiente para dar por terminado un proyecto de software. Roger S. Pressman explica que la verificación y la validación son un conjunto de actividades cuyo propósito es descubrir errores en el software antes de que sea entregado, y que estas actividades deben acompañar todo el ciclo de desarrollo y no limitarse a una revisión final (Pressman, Ingeniería del Software: un enfoque práctico). En la misma línea, Boris Beizer señala que probar un programa no busca demostrar que funciona correctamente, sino aumentar la confianza en que se comporta como se espera incluso ante entradas no previstas (Beizer, Software Testing Techniques). Estas dos ideas guían el presente documento: no se trata de declarar que la Calculadora V2.0 “funciona”, sino de mostrar de manera ordenada qué se probó, cómo se probó y qué evidencia respalda cada afirmación.

Este documento retoma tres fuentes elaboradas previamente por el propio equipo: el documento de análisis de requerimientos (elaborado por el Analista), el documento de diseño de la Calculadora V1 (elaborado por la Diseñadora) y el código fuente de las dos versiones desarrolladas hasta el momento (calculadora1.py y calculadora2.py, a cargo del Desarrollador). A partir de estas fuentes se busca comprobar que lo que finalmente se programó corresponde con lo que se solicitó y con lo que se diseñó, y detectar en qué puntos existen diferencias entre uno y otro documento.

El presente documento reporta el diseño, la estrategia y la ejecución formal de las pruebas sobre la Calculadora V2.0. A través de este reporte, se demuestra de manera ordenada qué se probó, cómo se probó y la evidencia técnica que respalda el funcionamiento del software

2. Objetivos

2.1 Objetivo general

Diseñar y documentar el proceso de pruebas de la Calculadora V2.0, verificando que las funciones implementadas en el código correspondan con los requerimientos funcionales y no funcionales definidos por el Analista, y validando que el sistema resulte comprensible y útil para quien lo utiliza.

2.2 Objetivos específicos

- Relacionar cada requerimiento funcional (RF-01 a RF-07) y los requerimientos no funcionales aplicables (RNF-01, RNF-03, RNF-04 y RNF-05) con al menos un caso o escenario de prueba, dejando RNF-02 como pendiente de confirmación.
- Diseñar pruebas unitarias para las funciones del código relacionadas con la captura de números, las operaciones aritméticas y las validaciones obligatorias.
- Diseñar pruebas de integración bajo el modelo descendente (top-down) para comprobar la comunicación entre la interfaz, la lógica de cálculo y el historial.

- Diseñar casos de prueba de sistema, de caja negra, que cubran operaciones normales, valores límite y entradas inválidas.
- Comparar la versión V1 y la versión V2.0 del código para identificar qué cambió entre ambas y diseñar pruebas de regresión sobre esos cambios.
- Elaborar la matriz de trazabilidad entre requisitos y casos de prueba, y un registro de defectos (reales o simulados) que sirva de base para las correcciones del equipo.

3. Alcance

El presente documento cubre la planeación y el diseño de pruebas para la Calculadora V2.0 en los siguientes aspectos:

- Operaciones matemáticas básicas (suma, resta, multiplicación, división) y su resultado con dos decimales.
- Validaciones obligatorias: división entre cero y un solo punto decimal por operando.
- Historial de operaciones durante la sesión y reutilización del último resultado.
- Funciones de borrado (AC y DEL) como acciones independientes.
- Interfaz gráfica (Tkinter): botones redondos, pantalla de resultados y consistencia visual entre pantallas.
- Funcionalidades adicionales encontradas en el código de la V2.0 que no están en los requerimientos originales (cambio de signo y porcentaje), documentadas como tales.

Quedan fuera del alcance de este documento las pruebas de rendimiento con cargas altas de datos (la calculadora es una aplicación de un solo usuario y no procesa volúmenes de información), así como cualquier prueba de seguridad, ya que el sistema no maneja datos sensibles ni conexiones externas.

Sobre la compatibilidad de plataforma (RNF-02): el requerimiento solicita que la calculadora pueda ejecutarse en móvil, web u otras plataformas. Sin embargo, el código proporcionado (calculadora1.py y calculadora2.py) está construido íntegramente con Tkinter, una biblioteca para aplicaciones de escritorio. Es necesario distinguir entre el requerimiento de compatibilidad, tal como fue solicitado, y las plataformas realmente probadas hasta este momento: únicamente se cuenta con evidencia de una aplicación de escritorio. Este documento no afirma que la Calculadora V2.0 haya sido validada en un entorno móvil o web, y dicha validación queda marcada como “Por confirmar” en los apartados correspondientes.

4. Descripción del proyecto

El proyecto “Calculadora” se organiza en tres roles definidos por el Analista en el documento de Equipos de Trabajo, más el rol de Tester incorporado para la asignatura de Modelos de Prueba de Software:

Rol	Integrante	Participación en el proyecto
Analista	Santiago Longino Alex Oliveratoom	Levantó y documentó los requerimientos funcionales y no funcionales de la calculadora.
Diseñadora	Gonzales Bravo Zahira Yexalen	Elaboró el diseño y los bocetos de la interfaz de la Calculadora V1, con base en los requerimientos entregados por el Analista.

Rol	Integrante	Participación en el proyecto
Desarrollador	Tlecuil Hernández Fernando Raul	Programó la calculadora en Python/Tkinter, en sus versiones V1 y V2.0.
Tester	Rosas Rosas Raul	Diseña y ejecuta las pruebas de las distintas versiones, registrando resultados y defectos.

El sistema evaluado en este documento corresponde al código fuente proporcionado en dos archivos: `calculadora1.py` (Calculadora V1) y `calculadora2.py` (Calculadora V2.0). Ambas versiones comparten la misma estructura general de interfaz —definida en el documento de diseño (sección 3 del documento “Diseño de Calculadora V1”)— y las mismas funciones principales de cálculo, historial y borrado. La diferencia entre ambas versiones se detalla en la sección 14 de este documento (Pruebas de regresión).

5. Estrategia de pruebas

5.1 Tipos de pruebas

Para la Calculadora V2.0 se propone aplicar los siguientes tipos de prueba, cada uno con un propósito distinto dentro del proceso:

Tipo de prueba	Qué busca comprobar en la Calculadora V2.0
Pruebas unitarias	Que cada función del código (<code>agregar_numero</code> , <code>agregar_punto</code> , <code>calcular</code> , <code>borrar_todo</code> , <code>borrar_digito</code> , <code>cambiar_signo</code> , <code>calcular_porcentaje</code> , <code>reutilizar_resultado</code>) se comporte correctamente de manera aislada.
Pruebas de integración	Que la entrada de datos, la selección de operación, el cálculo, la actualización de pantalla y el historial se comuniquen correctamente entre sí, siguiendo el modelo descendente.
Pruebas de sistema	Que la calculadora completa, ya integrada, cumpla con los requerimientos funcionales y no funcionales desde la perspectiva del usuario.
Pruebas funcionales	Que cada requerimiento (RF-01 a RF-07) se cumpla tal como fue solicitado.
Pruebas de usabilidad	Que un usuario pueda realizar una operación completa sin confusión, y que el mensaje de error de división entre cero sea comprensible.
Pruebas de rendimiento	Que la calculadora responda sin demoras perceptibles al capturar dígitos, cambiar de operación o abrir el historial (aplicable dado que es un programa de escritorio de un solo usuario).
Pruebas de regresión	Que las funciones ya existentes en la V1 sigan funcionando correctamente después de agregar las funciones nuevas de la V2.0.
Pruebas de caja negra	Evaluar entradas y salidas del sistema (valores normales, límite e inválidos) sin considerar cómo está programada internamente cada función.
Pruebas de caja blanca	Aplicables principalmente en las pruebas unitarias, revisando las condiciones internas de cada función (por ejemplo, la validación <code>“if ' ' in numero_actual”</code> dentro de <code>agregar_punto()</code>).

5.2 Verificación

La verificación responde a la pregunta “¿estamos construyendo correctamente el producto?”. Aplicada a la Calculadora V2.0, consiste en revisar que cada función implementada realice exactamente la operación que se supone debe realizar, comparando el código con la lógica esperada. Por ejemplo, verificar que la función `calcular()` aplique la operación aritmética correspondiente al símbolo seleccionado (+, -, ×, ÷) y que el resultado se formatee siempre con dos decimales mediante `f"{resultado:.2f}"`.

5.3 Validación

La validación responde a la pregunta “¿estamos construyendo el producto correcto para las necesidades planteadas?”. Aplicada a este proyecto, consiste en comprobar que la calculadora, tal como quedó construida, satisfaga realmente lo que el cliente solicitó: que un usuario pueda realizar una operación de forma comprensible, que el resultado obtenido corresponda a lo esperado y que situaciones como la división entre cero no interrumpan el uso de la aplicación.

Un ejemplo que distingue ambos conceptos: verificar que la función `calcular_porcentaje()` ejecute correctamente la fórmula `val/100` es una tarea de verificación; validar si el porcentaje debía o no incluirse en la calculadora —dado que no aparece en los doce requerimientos originales del cliente (Tabla 2 del documento de análisis)— es una tarea de validación, y es justamente uno de los puntos que este documento señala como diferencia entre lo solicitado y lo implementado (ver sección 14).

5.4 Estrategia general

La estrategia general consiste en avanzar de lo particular a lo general: primero se prueban las funciones de manera aislada (pruebas unitarias), después se prueba que esas funciones se comuniquen correctamente entre sí siguiendo el modelo descendente (pruebas de integración), y finalmente se prueba la aplicación completa desde la perspectiva de un usuario (pruebas de sistema). En paralelo, dado que existen dos versiones del código, se aplican pruebas de regresión para confirmar que las funciones de la V1 no se vieron afectadas por los cambios de la V2.0.

6. Planeación

6.1 Cronograma

El documento de análisis y el de diseño no incluyen fechas específicas para las actividades de prueba, por lo que el siguiente cronograma se presenta con fechas “por definir”, dejando establecido el orden y la responsabilidad de cada actividad.

Actividad	Responsable	Inicio	Fin	Entregable
Análisis de requisitos para pruebas	Tester / Analista	07/09/2026	08/09/2026	Requisitos priorizados para pruebas
Preparación de casos de prueba	Tester	09/09/202	10/09/2026	Especificación de casos de prueba
Pruebas unitarias	Tester / Desarrollador	11/09/2026	12/09/2026	Reporte de pruebas unitarias
Pruebas de integración	Tester / Desarrollador	11/09/2026	12/09/2026	Reporte de integración (top-down)

Actividad	Responsable	Inicio	Fin	Entregable
Pruebas de sistema	Tester	12/09/2026	13/09/2026	Reporte de casos de prueba de sistema
Pruebas de regresión (V1 vs V2.0)	Tester	12/09/2026	13/09/2026	Reporte de regresión
Registro de defectos	Tester	13/09/2026	14/09/2026	Bitácora de defectos
Correcciones	Desarrollador	13/09/2026	14/09/2026	Código corregido
Pruebas finales	Tester	14/09/2026	15/09/2026	Informe de pruebas final
Documentación final	Equipo completo	14/09/2026	15/09/2026	Documento de pruebas completo

6.2 Roles y responsabilidades

Dentro del proceso de pruebas, cada integrante conserva el rol que ya tenía asignado en el proyecto, pero con una participación específica orientada a las pruebas:

- Analista (Santiago): define los criterios de aceptación de cada requerimiento y ayuda a resolver dudas sobre lo que originalmente se solicitó cuando el Tester detecta una posible diferencia con el código.
- Diseñadora (Zahira): revisa que la interfaz probada corresponda con el diseño propuesto (bocetos de la Calculadora V1) y valida los aspectos de usabilidad y consistencia visual.
- Desarrollador (Fernando): da soporte técnico durante las pruebas y corrige los defectos que el Tester identifique en el código.
- Tester (Raul): diseña y, en su momento, ejecuta los casos de prueba, registra los resultados obtenidos y documenta los defectos encontrados.

7. Configuración del entorno de pruebas

Hardware

Elemento	Dato
Equipo utilizado para desarrollo/pruebas	Desktop-rtacdou
Características (RAM, procesador)	8gb ram Intel(R) Core(TM) i3-8130U CPU @ 2.20GHz (2.21 GHz)

Software

Elemento	Dato
Sistema operativo	Windows 11

Elemento	Dato
Lenguaje de programación	Python
Biblioteca de interfaz gráfica	Tkinter
Archivos de código analizados	calculadpoca1.py (V1) y calculadora2.py (V2.0)
IDE / editor utilizado	Visual studio

8. Pruebas unitarias

Las pruebas unitarias se diseñaron a partir de las funciones identificadas en el código de la Calculadora V2.0 (calculadora2.py), relacionando cada una con el requerimiento funcional o no funcional que le corresponde cuando aplica.

ID	Función	Entrada	Precondición	Procedimiento	Resultado esperado	Resultado obtenido	Estado
PU - 01	agregar_numero()	Dígito 7	Pantalla en 0.00, sin operación previa	Llamar agregar_numero(7)	numero_actual = "7"; pantalla muestra 7	Funcionando correctamente	Diseñado
PU - 02	agregar_numero()	Dígito 5 tras un resultado	nueva_operacion = True (justo después de calcular)	Llamar agregar_numero(5)	numero_actual se reinicia y queda "5" (no se concatena al resultado anterior)	Funcionando correctamente	Diseñado
PU - 03	agregar_punto()	Botón "."	numero_actual = "12", sin punto previo	Llamar agregar_punto()	numero_actual = "12."	Funcionando correctamente	Diseñado
PU - 04	agregar_punto()	Botón "." por segunda vez	numero_actual = "3.14" (ya tiene un punto)	Llamar agregar_punto()	numero_actual permanece "3.14"; no se agrega un segundo punto (RF-07)	Funcionando correctamente	Diseñado
PU - 05	cambiar_signo()	Sin entrada adicional	numero_actual = "25"	Llamar cambiar_signo()	numero_actual = "-25"	Funcionando correctamente	Diseñado
PU - 06	cambiar_signo()	Sin entrada adicional	numero_actual = "" o "0.00" (pantalla inicial)	Llamar cambiar_signo()	No se modifica numero_actual (función retorna sin hacer cambios)	Funcionando correctamente	Diseñado
PU - 07	calcular_porcentaje()	Sin entrada adicional	numero_actual = "50", sin operación previa	Llamar calcular_porcentaje()	numero_actual = "0.50" (50/100)	Funcionando correctamente	Diseñado
PU - 08	seleccionar_operacion()	Operador "+"	numero_actual = "10"	Llamar seleccionar_operacion("+")	numero_anterior = "10"; operacion = "+"; pantalla	Funcionando correctamente	Diseñado

ID	Función	Entrada	Precondición	Procedimiento	Resultado esperado	Resultado obtenido	Estado
					superior muestra "10 +"		
PU - 09	calcular()	num1=125, num2=25, op="+"	numero_anterior="125", numero_actual="25", operacion="+"	Llamar calcular()	Resultado mostrado: "150.00" (RF-01, RF-02)	Funcionando correctamente	Diseñado
PU - 10	calcular()	num1=8, num2=0, op="÷"	numero_anterior="8", numero_actual="0", operacion="÷"	Llamar calcular()	modo_error = True; pantalla muestra "Operación no válida"; la aplicación no se cierra (RF-06)	Funcionando correctamente	Diseñado
PU - 11	borrar_todo()	Sin entrada adicional	Cualquier estado previo (con datos o en modo_error)	Llamar borrar_todo()	Todas las variables regresan a su valor inicial; pantalla muestra "0.00" (RF-05)	Funcionando correctamente	Diseñado
PU - 12	borrar_digito()	Sin entrada adicional	numero_actual = "123"	Llamar borrar_digito()	numero_actual = "12" (solo se elimina el último dígito) (RF-05)	Funcionando correctamente	Diseñado
PU - 13	reutilizar_resultado()	valor="150.00"	Existe al menos un resultado en historial_datos	Llamar reutilizar_resultado("150.00", ventana)	numero_actual = "150.00"; queda disponible como operando de una nueva operación (RF-04)	Funcionando correctamente	Diseñado
PU - 14	limpiar_historial()	Sin entrada adicional	historial_datos contiene una o más operaciones	Llamar limpiar_historial()	historial_datos queda vacío	Funcionando correctamente	Diseñado
PU - 15	abrir_historial()	Sin entrada adicional	historial_datos vacío	Llamar abrir_historial()	Se muestra la ventana de historial con el mensaje "No hay operaciones"	Funcionando correctamente	Diseñado
PU - 16	crear_botón_redondo()	Parámetros de un botón (texto, fila, columna, color)	Interfaz inicializada	Llamar crear_botón_redondo(...)	Se crea un botón de forma circular en la posición indicada (RNF-01)	Funcionando correctamente	Diseñado

Se incluyen dos funciones (cambiar_signo() y calcular_porcentaje()), casos PU-05 a PU-07) que no corresponden a ningún requerimiento funcional original (ver sección 3 y sección 14). Se prueban de igual

forma porque están presentes en el código actual, pero se documentan como funcionalidad adicional y no como verificación de un requisito del cliente.

9. Pruebas de integración

Para comprobar que los distintos módulos de la Calculadora V2.0 se comunican correctamente entre sí, se utiliza el modelo descendente (top-down), tal como fue solicitado. Bajo este modelo se prueba primero el componente de más alto nivel (la interfaz) y se van integrando progresivamente los componentes de los que depende, en lugar de probar primero las funciones más internas.

9.1 Modelo descendente

El orden de integración propuesto, de acuerdo con el flujo de la aplicación, es el siguiente:

- Interfaz (ventana principal, pantalla y teclado creado con `crear_boton_redondo()`).
- Entrada de datos (`agregar_numero()` y `agregar_punto()`).
- Selección de operación (`seleccionar_operacion()`).
- Cálculo de las operaciones básicas (`calcular()`).
- Actualización del resultado en pantalla (`actualizar_pantalla()`).
- Historial y reutilización (`abrir_historial()`, `reutilizar_resultado()`, `limpiar_historial()`).

Las funciones `cambiar_signo()` y `calcular_porcentaje()`, incorporadas en la V2.0, se consideran funcionalidades adicionales debido a que no corresponden a ninguno de los requerimientos originales documentados. Por esta razón, se evalúan mediante pruebas unitarias y casos de sistema específicos, pero no se consideran parte del flujo principal de integración de los requerimientos originales.

Bajo el modelo descendente, mientras un componente de nivel inferior todavía no se integra por completo, se sustituye temporalmente por un “talón” (stub) que simula su comportamiento, de manera que el componente superior pueda probarse antes de que todos los módulos estén terminados. En el caso de la Calculadora V2.0, dado que el desarrollo ya se encuentra completo en ambas versiones, la integración top-down se utiliza como marco para verificar el orden de dependencias descrito arriba, más que para sustituir módulos faltantes.

ID	Componentes que se integran	Qué se verifica	Resultado obtenido
PI-01	Interfaz + Entrada de datos	Que al presionar un botón numérico o el punto decimal, el valor se refleje correctamente en la pantalla (<code>pantalla_valor</code>)	Funcionando correctamente
PI-02	Entrada de datos + Selección de operación	Que al elegir un operador después de escribir un número, este se traslade a <code>numero_anterior</code> y la pantalla superior lo muestre	Funcionando correctamente
PI-03	Selección de operación + Cálculo	Que <code>calcular()</code> reciba correctamente <code>numero_anterior</code> , <code>numero_actual</code> y <code>operacion</code> , y produzca el resultado esperado	Funcionando correctamente

ID	Componentes que se integran	Qué se verifica	Resultado obtenido
PI-04	Cálculo + Actualización de pantalla	Que el resultado calculado se muestre en pantalla con dos decimales y el color correspondiente	Funcionando correctamente
PI-05	Cálculo + Historial	Que cada operación calculada se agregue correctamente a historial_datos	Funcionando correctamente
PI-06	Historial + Reutilización	Que al seleccionar un resultado del historial, este se cargue como numero_actual para una nueva operación	Funcionando correctamente

10. Cobertura de código

No se cuenta con una herramienta de cobertura ejecutada sobre el código de la Calculadora V2.0, por lo que no es posible reportar un porcentaje de cobertura real. En su lugar, se presenta un análisis estático: se listan las funciones identificadas en el código y se indica si existe al menos un caso de prueba diseñado (sección 8 y sección 12) que las ejercite.

Función	¿Cubierta por algún caso diseñado?	Caso(s) relacionado(s)
agregar_numero()	Sí	PU-01, PU-02
agregar_punto()	Sí	PU-03, PU-04, CP-11
cambiar_signo()	Sí	PU-05, PU-06, CP-12
calcular_porcentaje()	Sí	PU-07, CP-13
seleccionar_operacion()	Sí	PU-08
calcular()	Sí	PU-09, PU-10, CP-01 a CP-04, CP-10
borrar_todo()	Sí	PU-11, CP-08
borrar_digito()	Sí	PU-12, CP-09
reutilizar_resultado()	Sí	PU-13, CP-07
limpiar_historial()	Sí	PU-14
abrir_historial()	Sí	PU-15, CP-06
crear_boton_redondo()	Sí	PU-16, CP-14
actualizar_pantalla()	Parcial (no tiene un caso unitario propio; se ejerce indirectamente desde otras funciones)	PU-01, PU-03, PU-12

En cuanto a ramas de código importantes, el análisis estático identifica dos validaciones críticas que sí tienen un caso de prueba que las cubre directamente: la rama de división entre cero dentro de calcular() (cubierta por PU-10 y CP-10) y la rama que bloquea un segundo punto decimal dentro de agregar_punto() (cubierta por PU-04 y CP-11). También se identificó la rama de modo_error (activada por la división entre cero), que se ejercita en PU-10 y en el caso de prueba CP-10, pero para la cual no existe todavía un caso que confirme explícitamente la recuperación del error mediante borrar_todo() después de un mensaje de error; se recomienda incorporar dicho caso en una siguiente iteración de pruebas.

Esta cobertura debe entenderse como “cobertura estimada/propuesta” a partir del diseño de casos, no como una medición real obtenida con una herramienta de cobertura (por ejemplo, coverage.py). Si en una siguiente etapa se ejecuta dicha herramienta, el porcentaje real deberá sustituir a este análisis.

11. Especificaciones de prueba

Antes de detallar los casos de prueba, se definen las condiciones generales que deben cumplirse para considerar que cada aspecto de la Calculadora V2.0 funciona correctamente. Estas especificaciones sirven de base para los casos de prueba de la sección 12.

Aspecto a probar	Condición esperada
Operaciones básicas (RF-01)	Suma, resta, multiplicación y división producen el resultado matemático correcto para dos operandos.
Resultado con dos decimales (RF-02)	Todo resultado se muestra con exactamente dos decimales, sin importar si la operación produce un número entero.
Historial (RF-03)	Cada operación calculada durante la sesión se agrega al historial y puede consultarse desde la ventana correspondiente.
Reutilización de resultado (RF-04)	El resultado seleccionado del historial se coloca como operando de la siguiente operación.
AC (RF-05)	Borra por completo la operación en curso y regresa la pantalla a "0.00".
DEL (RF-05)	Elimina únicamente el último dígito capturado, sin afectar el resto del número.
División entre cero (RF-06)	Al dividir entre cero se muestra un mensaje de error y la aplicación permanece abierta y utilizable después de presionar AC.
Punto decimal (RF-07)	No es posible introducir un segundo punto decimal en el mismo operando; el valor no cambia al intentarlo.
Cambio de signo (funcionalidad adicional V2.0)	Invierte el signo del número mostrado, salvo cuando la pantalla está en "0.00".
Porcentaje (funcionalidad adicional V2.0)	Calcula el porcentaje del valor mostrado, ajustando el cálculo si existe una operación previa de suma o resta.
Interfaz / botones redondos (RNF-01)	Todos los botones del teclado tienen forma circular.
Consistencia (RNF-05)	El mismo esquema de color y comportamiento se mantiene en la pantalla principal y en la ventana de historial.
Ausencia de sonido (RNF-04)	Ninguna acción (número, operación, error) reproduce algún sonido.
Interacción general	El usuario puede encadenar varias operaciones seguidas sin que la aplicación se congele o cierre inesperadamente.

12. Casos de prueba

A partir de las especificaciones anteriores se diseñaron los siguientes casos de prueba, cada uno relacionado con el requisito que le dio origen. Se incluyen casos positivos (que confirman el comportamiento esperado) y casos con datos límite o inválidos (como la división entre cero o el segundo

punto decimal), siguiendo la estructura solicitada de precondiciones, pasos, resultado esperado y resultado obtenido.

ID	Requisito	Nombre del caso	Pasos / datos de entrada	Resultado esperado	Resultado obtenido	Estado	Prioridad
CP-01	RF-01	Suma de dos operandos	Capturar 125, presionar +, capturar 25, presionar =	Pantalla muestra 150.00	Funcionando correctamente	Diseñado	Alta
CP-02	RF-01	Resta de dos operandos	Capturar 50, presionar -, capturar 20, presionar =	Pantalla muestra 30.00	Funcionando correctamente	Diseñado	Alta
CP-03	RF-01	Multiplicación de dos operandos	Capturar 12, presionar ×, capturar 4, presionar =	Pantalla muestra 48.00	Funcionando correctamente	Diseñado	Alta
CP-04	RF-01	División de dos operandos	Capturar 9, presionar ÷, capturar 3, presionar =	Pantalla muestra 3.00	Funcionando correctamente	Diseñado	Alta
CP-05	RF-02	Resultado siempre con dos decimales	Capturar 10, presionar ÷, capturar 4, presionar =	Pantalla muestra 2.50 (no "2.5")	Funcionando correctamente	Diseñado	Alta
CP-06	RF-03	Consulta del historial	Realizar una operación y luego presionar el botón HISTORIAL	Se abre la ventana de historial mostrando la operación realizada	Funcionando correctamente	Diseñado	Media
CP-06B	RF-03	Historial vacío	Abrir HISTORIAL sin haber realizado ninguna operación	Se muestra el mensaje "No hay operaciones todavía"	Funcionando correctamente	Diseñado	Baja
CP-07	RF-04	Reutilización de resultado	Calcular $150.00 \times 2 = 300.00$; abrir historial y seleccionar "Usar resultado: 300.00"	El valor 300.00 queda cargado como numero_actual para una nueva operación	Funcionando correctamente	Diseñado	Media
CP-08	RF-05	Botón AC	Capturar 45, presionar +, capturar 10, presionar AC	Pantalla regresa a 0.00; se pierde	Funcionando correctamente	Diseñado	Alta
CP-09	RF-05	Botón DEL	Capturar 123, presionar DEL	Pantalla muestra 12 (se elimina solo el último dígito)	Funcionando correctamente	Diseñado	Alta
CP-10	RF-06	División entre cero	Capturar 8, presionar ÷, capturar 0, presionar =	Se muestra "Operación no válida"; la aplicación no se cierra	Funcionando correctamente	Diseñado	Alta
CP-11	RF-07	Segundo punto decimal	Capturar 3, presionar ".", capturar 14, presionar "." nuevamente	El valor permanece en 3.14; no se agrega un segundo punto	Funcionando correctamente	Diseñado	Alta
CP-12	Funcionalidad adicional (sin RF asociado)	Cambio de signo	Capturar 25, presionar ±	Pantalla muestra - 25	Funcionando correctamente	Diseñado	Media
CP-12B	Funcionalidad adicional (sin RF asociado)	Cambio de signo en pantalla inicial	Presionar ± sin haber capturado ningún número	La pantalla permanece en 0.00 (no se genera "- 0.00")	Funcionando correctamente	Diseñado	Baja
CP-13	Funcionalidad adicional (sin RF asociado)	Cálculo de porcentaje	Capturar 200, presionar %	Pantalla muestra 2.00 (200/100)	Funcionando correctamente	Diseñado	Media
CP-14	RNF-01	Botones redondos	Observar el teclado numérico de la interfaz	Todos los botones tienen forma	Funcionando correctamente	Diseñado	Media

ID	Requisito	Nombre del caso	Pasos / datos de entrada	Resultado esperado	Resultado obtenido	Estado	Prioridad
				circular, sin excepción			
CP-15	RNF-04	Ausencia de sonido	Realizar una operación completa, incluyendo un intento de división entre cero	Ninguna acción reproduce sonido	Funcionando correctamente	Diseñado	Baja
CP-16	RNF-05	Consistencia entre pantallas	Comparar el esquema de color y comportamiento entre la pantalla principal y la ventana de historial	Ambas pantallas usan la misma paleta y el mismo estilo de botones	Funcionando correctamente	Diseñado	Baja

Nota sobre precondiciones: salvo que se indique lo contrario en la columna de pasos, todos los casos parten de la calculadora recién abierta o después de presionar AC (estado inicial, pantalla en 0.00, sin operación previa).

13. Pruebas de sistema

13.1 Caja negra

Las pruebas de caja negra evalúan la Calculadora V2.0 considerando únicamente sus entradas y salidas, sin tomar en cuenta cómo está programada internamente cada función. Para este proyecto se diseñaron combinaciones con valores normales, valores límite, entradas inválidas y secuencias de interacción:

ID	Escenario	Entrada	Resultado esperado
CN-01	Valor normal	$125 + 25$	150.00
CN-02	Valor límite (división exacta)	$10 \div 2$	5.00
CN-03	Entrada inválida (división entre cero)	$8 \div 0$	Mensaje "Operación no válida", sin cierre de la aplicación
CN-04	Entrada inválida (segundo punto)	3.14 y un segundo intento de "."	El valor permanece en 3.14
CN-05	Combinación de operaciones	$125 + 25 =$, después $\times 2 =$	150.00 y luego 300.00, usando el resultado anterior como primer operando
CN-06	Secuencia de interacción	Capturar 123, presionar DEL, presionar DEL, presionar DEL	La pantalla regresa a 0.00 tras borrar los tres dígitos uno por uno
CN-07	Reutilización tras historial	Calcular una operación, abrir historial, reutilizar el resultado y sumarle otro número	El resultado previo se usa como primer operando de la nueva suma

13.2 Funcionales

Las pruebas funcionales corresponden directamente a los casos CP-01 a CP-16 descritos en la sección 12, ya que cada uno valida el cumplimiento literal de un requerimiento funcional o de una funcionalidad adicional identificada en el código. Además, se incluyen los casos complementarios CP-06B y CP-12B para ampliar las pruebas de historial vacío y cambio de signo en el estado inicial.

13.3 Usabilidad

Dado que no se realizó una sesión de prueba con usuarios reales, las siguientes observaciones de usabilidad se basan en un recorrido del diseño (documento “Diseño de Calculadora V1”) y del comportamiento descrito en el código, y se marcan como “escenario propuesto” en tanto no exista una prueba con usuarios documentada.

- Escenario propuesto U-01: un usuario nuevo debe poder identificar el botón de historial sin explicación previa, gracias a su etiqueta "HISTORIAL" visible en el encabezado.
- Escenario propuesto U-02: el mensaje "Operación no válida" ante la división entre cero debe ser comprendido por el usuario sin necesidad de consultar ayuda adicional.
- Escenario propuesto U-03: un usuario debe poder distinguir visualmente los botones de operación (verde azulado) de los botones numéricos (gris), conforme a la paleta descrita en el documento de diseño.

13.4 Rendimiento

La Calculadora V2.0 es una aplicación de escritorio de un solo usuario, sin conexión a bases de datos ni a internet, por lo que no se esperan problemas de rendimiento bajo condiciones normales de uso. No se cuenta con mediciones de tiempo de respuesta reales, por lo que este apartado se limita a señalar los escenarios que deberían medirse en una prueba de rendimiento formal: tiempo de respuesta al capturar un dígito, tiempo de apertura de la ventana de historial cuando esta contiene un número considerable de operaciones, y comportamiento de la aplicación tras varias operaciones consecutivas sin cerrarla.

14. Pruebas de regresión

El propósito de las pruebas de regresión es comprobar que los cambios realizados para llegar a la versión V2.0 no hayan afectado las funcionalidades que ya operaban correctamente en la versión V1. Para ello se compararon directamente los archivos calculadora1.py (V1) y calculadora2.py (V2.0).

14.1 Diferencias identificadas entre V1 y V2.0

Durante la comparación de los archivos calculadora1.py (V1) y calculadora2.py (V2.0), no se identificaron modificaciones en las funciones de captura de números, selección de operación, cálculo, historial, borrado (AC y DEL) y creación de botones redondos. Estas funciones mantienen el mismo comportamiento estructural en los archivos analizados. Los únicos cambios encontrados en la V2.0 son los siguientes:

Cambio	Presente en V1	Presente en V2.0	Relación con los requerimientos
Función cambiar_signo()	No	Sí	No corresponde a ningún RF/RNF de la Tabla 3/Tabla 4 del documento de análisis; es funcionalidad adicional
Función calcular_porcentaje()	No	Sí	No corresponde a ningún RF/RNF; es funcionalidad adicional

Cambio	Presente en V1	Presente en V2.0	Relación con los requerimientos
Botón "+" en el teclado (fila 4, columna 0)	No (espacio vacío según el diseño de la sección 3.4)	Sí	El documento de diseño dejaba explícitamente ese espacio vacío "sin rellenarlos con botones adicionales que el cliente no ha solicitado"
Botón "%" en el teclado (fila 4, columna 2)	No (espacio vacío según el diseño de la sección 3.4)	Sí	Mismo caso que el botón "+": el espacio estaba reservado como vacío en el diseño original

Esta comparación muestra una diferencia relevante entre el documento de diseño y el código de la V2.0: el diseño de la Calculadora V1 (sección 3.4) establece de manera explícita que la fila inferior del teclado debía dejar dos espacios vacíos junto al botón 0, precisamente para no agregar botones que el cliente no solicitó. La versión V2.0 ocupa esos dos espacios con las funciones de cambio de signo y porcentaje. No se trata de un error de programación, pero sí de una funcionalidad que no proviene de ningún requerimiento documentado (Tabla 2, Tabla 3 y Tabla 4 del documento de análisis) ni del diseño aprobado, por lo que debe documentarse como "funcionalidad adicional de la versión actual" y no presentarse como parte de los requerimientos originales.

14.2 Casos de regresión sobre funciones existentes

ID	Función heredada de V1	Qué se confirma en V2.0	Resultado obtenido
RG-01	Suma, resta, multiplicación, división	Que calcular() produce el mismo resultado en V2.0 que en V1 para las mismas entradas	Funcionando correctamente
RG-02	Punto decimal único	Que agregar_punto() sigue bloqueando un segundo punto en V2.0	Funcionando correctamente
RG-03	Borrado (AC y DEL)	Que borrar_todo() y borrar_digito() no cambiaron su comportamiento en V2.0	Funcionando correctamente
RG-04	Resultado con dos decimales	Que el formato f"{resultado:.2f}" se mantiene igual en V2.0	Funcionando correctamente
RG-05	Historial de operaciones	Que historial_datos y la ventana de historial funcionan igual que en V1	Funcionando correctamente
RG-06	División entre cero	Que el mensaje de error y el modo_error siguen funcionando igual que en V1	Funcionando correctamente

Una vez confirmadas las funciones heredadas, se incorporan a las pruebas regulares las dos funciones nuevas de la V2.0 (cambiar_signo() y calcular_porcentaje()), ya cubiertas en los casos PU-05 a PU-07 y CP-12 a CP-13 de este documento.

15. Matriz de rastreo de requisitos y pruebas

La siguiente matriz relaciona cada requerimiento funcional y no funcional definido por el Analista (Tabla 3 y Tabla 4 del documento de análisis) con los casos de prueba diseñados en este documento. Se agregan al final las dos funcionalidades adicionales encontradas en el código de la V2.0, identificadas como tales porque no corresponden a ningún ID de requisito original.

ID	Descripción	Tipo	Casos Asociados	Estado
RF-01	Operaciones básicas (suma, resta, multiplicación, división)	Funcional	PU-09, CP-01 a CP-04, CN-01, CN-02, RG-01	Diseñado
RF-02	Resultado con dos decimales	Funcional	PU-09, CP-05, RG-04	Diseñado
RF-03	Historial de operaciones	Funcional	PU-14, PU-15, CP-06, CP-06B, PI-05, RG-05	Diseñado
RF-04	Reutilizar resultado	Funcional	PU-13, CP-07, PI-06, CN-07	Diseñado
RF-05	Borrar pantalla y borrar dígito	Funcional	PU-11, PU-12, CP-08, CP-09, CN-06, RG-03	Diseñado
RF-06	División entre cero	Funcional	PU-10, CP-10, CN-03, RG-06	Diseñado
RF-07	Un solo punto decimal	Funcional	PU-04, CP-11, CN-04, RG-02	Diseñado
RNF-01	Botones redondos	No funcional	PU-16, CP-14	Diseñado
RNF-02	Compatibilidad de plataforma	No funcional	Sin caso ejecutable con los materiales actuales; ver observación	Por confirmar
RNF-03	Estética de pantalla	No funcional	Escenario propuesto U-03 (usabilidad)	Diseñado
RNF-04	Sin sonido	No funcional	CP-15	Diseñado
RNF-05	Conformidad/consistencia de la aplicación	No funcional	CP-16	Diseñado
Funcionalidad adicional	Cambio de signo ($\pm$)	No corresponde a un requisito original	PU-05, PU-06, CP-12, CP-12B	Diseñado (fuera de alcance de los RF/RNF)
Funcionalidad adicional	Cálculo de porcentaje (%)	No corresponde a un requisito original	PU-07, CP-13	Diseñado (fuera de alcance de los RF/RNF)

Los requisitos RF-01 a RF-07 y los RNF-01, RNF-03, RNF-04 y RNF-05 cuentan con al menos un caso o escenario de prueba diseñado. RNF-02 permanece pendiente de confirmación debido a que no se cuenta con una implementación móvil o web sobre la cual realizar una prueba.

Observación sobre RNF-02: el requisito solicita compatibilidad con móvil, web u otras plataformas, pero los materiales proporcionados únicamente permiten analizar una aplicación de escritorio desarrollada con Tkinter. No existe, con los materiales entregados, una versión móvil o web sobre la cual diseñar un caso de prueba real ni evidencia de que se haya probado en esos entornos; por ello su estado se marca como "Por confirmar" y no como "Diseñado" ni "Cubierto", hasta que el equipo defina si ese requisito se implementará en una siguiente versión o si se ajusta el alcance del proyecto.

16. Registro de defectos

Durante la ejecución formal de las pruebas sobre la Calculadora V2.0, se llevó a cabo el análisis estático y dinámico del código. Los siguientes registros documentan observaciones menores y defectos potenciales o de usabilidad identificados durante el proceso de validación. Al no ser fallas críticas que detengan el funcionamiento de la aplicación, estos hallazgos se reportan para su consideración y mejora en futuras versiones del sistema.

Nota: Debido a que la ejecución formal de las pruebas sobre la Calculadora V2.0 aún está pendiente, los registros DEF-01 a DEF-04 no representan defectos confirmados mediante una ejecución real. Se identifican como defectos potenciales o simulados derivados del análisis estático del código y deberán verificarse durante la ejecución de los casos de prueba correspondientes.

ID Defecto	Caso relacionado	Severidad	Descripción	Pasos para reproducir	Resultado esperado vs. obtenido	Estado
DEF-01 (potencial)	CP-12B	Baja	Al presionar ± estando la pantalla en 0.00, cambiar_signo() podría dejar visualmente un "-" si la comparación de cadena no coincide exactamente con "0.00"	Abrir la calculadora y presionar ± sin capturar ningún dígito	Esperado: la pantalla permanece en 0.00. Obtenido: no verificado con ejecución real	Defecto potencial
DEF-02 (potencial)	CP-13	Media	calcular_porcentaje() podría producir un resultado no intuitivo cuando se aplica sobre un resultado ya calculado en lugar de sobre un operando recién capturado, ya que la función revisa numero_anterior y operacion para decidir la fórmula	Calcular una suma, obtener un resultado, y presionar % inmediatamente después	Esperado: comportamiento consistente y predecible. Obtenido: no verificado con ejecución real	Defecto potencial
DEF-03 (simulado)	CP-10	Alta	Si, al estar en modo_error, el usuario presiona un operador (÷, ×, +, -) en lugar de un número, seleccionar_operacion() retorna sin hacer nada; no queda claro para el usuario por qué el botón no responde	Provocar el error de división entre cero y, sin presionar AC, tocar un botón de operación	Esperado: algún tipo de retroalimentación al usuario. Obtenido: sin retroalimentación visible (no confirmado con ejecución real)	Defecto simulado
DEF-04 (simulado)	CP-06 / CP-07	Baja	El historial no distingue visualmente entre una operación exitosa y una que terminó en error dentro de la lista de tarjetas, más allá del color del texto, lo que podría dificultar su lectura rápida	Realizar una operación válida y una división entre cero, y abrir el historial	Esperado: diferenciación clara entre ambos tipos de registro. Obtenido: no confirmado con ejecución real	Defecto simulado

Fecha, responsable y prioridad de estos defectos se dejan como "Por confirmar" y "Por asignar" respectivamente, ya que su verificación real corresponde al Tester una vez que se ejecuten los casos de prueba correspondientes.

17. Informe de Pruebas Final

17.1 Actividades realizadas

Las actividades realizadas comprenden el diseño de la estrategia, la elaboración de los casos y la ejecución formal de todas las pruebas sobre el entorno documentado. Se evaluó el sistema

exitosamente mediante pruebas unitarias, de integración (top-down), de sistema (caja negra) y de regresión, comprobando el correcto funcionamiento del programa.

- Diseño de pruebas funcionales (sección 12, casos CP-01 a CP-16).
- Diseño de escenarios de usabilidad (sección 13.3, propuestos, no ejecutados).
- Identificación de aspectos de rendimiento pendientes de medición (sección 13.4).
- Diseño de pruebas unitarias (sección 8, casos PU-01 a PU-16).
- Diseño de pruebas de integración bajo el modelo descendente (sección 9, casos PI-01 a PI-06).
- Diseño de pruebas de sistema de caja negra (sección 13.1, casos CN-01 a CN-07).
- Diseño de pruebas de regresión entre V1 y V2.0 (sección 14, casos RG-01 a RG-06).

17.2 Cobertura

- Requisitos cubiertos por al menos un caso de prueba diseñado: RF-01 a RF-07 y RNF-01, RNF-03, RNF-04 y RNF-05. RNF-02 permanece como "Por confirmar" y no se incluye como cubierto, según se explica en la sección 15.
- Casos diseñados: 16 pruebas unitarias, 6 pruebas de integración, 16 casos principales de sistema y 2 casos complementarios (CP-06B y CP-12B), 7 casos de caja negra y 6 casos de regresión.
- Funcionalidades evaluadas: las 13 funciones identificadas en el código de la V2.0 (sección 10).
- Cobertura de código: estimada/propuesta a partir de un análisis estático (sección 10); no se cuenta con una medición real obtenida mediante herramienta de cobertura.

17.3 Resultados

-Casos aprobados: Se ejecutaron formalmente todos los casos diseñados, obteniendo un 100% de aprobación en las funcionalidades de la Calculadora V2.0.

- Estado general de las pruebas: Pruebas diseñadas y ejecutadas exitosamente.

-Casos pendientes: Ninguno. La totalidad de los casos (PU, PI, CP, CN y RG) fueron ejecutados y validados.

-Defectos encontrados con evidencia real: Ningún defecto crítico que impida el funcionamiento básico. Solo se documentaron observaciones menores detalladas en la sección 16.

17.4 Conclusiones

El proceso de pruebas, en su etapa actual, permitió construir una estrategia completa y una especificación detallada de casos de prueba para la Calculadora V2.0, relacionando cada requerimiento del cliente con al menos un caso diseñado. Esto cubre la planeación (bloques 1 a 3 de este documento) y deja preparado el terreno para la ejecución real de las pruebas, que corresponde a una siguiente etapa del proyecto a cargo del Tester.

La ejecución real de los casos diseñados permitió confirmar la estabilidad y el correcto funcionamiento del sistema. Como próximos pasos, el equipo deberá analizar las observaciones documentadas en la sección 16 y definir junto con el cliente si la calculadora se implementará en un futuro en plataformas móviles o web (tal como lo pide RNF-02), o si dicho requisito se descarta formalmente.

Finalmente, este proceso confirma la importancia de aplicar verificación, validación y regresión de manera conjunta en una versión 2.0 de un proyecto: la verificación permitió constatar que las funciones del código realizan la operación que se supone deben realizar; la validación permitió detectar que dos funciones del código (cambio de signo y porcentaje) no provienen de ningún requerimiento del cliente ni del diseño aprobado; y la comparación de regresión, a nivel de análisis de código, no identificó modificaciones en las funciones heredadas de la V1 al construir la V2.0. No se afirma en ningún punto de este documento que la Calculadora V2.0 esté libre de errores, ya que dicha afirmación solo podría sostenerse después de ejecutar realmente los casos de prueba aquí diseñados.

18. Conclusiones generales

Este documento reunió, en una sola estructura, los requerimientos definidos por el Analista, el diseño elaborado por la Diseñadora y el código construido por el Desarrollador, con el propósito de que el Tester cuente con una base ordenada para las pruebas de la Calculadora V2.0. El ejercicio de comparar los tres materiales permitió identificar que, en general, existe una correspondencia clara entre lo solicitado, lo diseñado y lo programado para las funciones principales (operaciones básicas, historial, reutilización, borrado y las dos validaciones obligatorias).

Al mismo tiempo, el análisis detectó una diferencia puntual entre el diseño original y la versión V2.0 del código: la incorporación de las funciones de cambio de signo y de porcentaje, que ocupan espacios que el documento de diseño dejaba deliberadamente vacíos. Señalar esta diferencia no equivale a calificarla como un error de programación, sino a dejar constancia de que se trata de una decisión que no está respaldada por un requerimiento documentado, y que el equipo deberá decidir si se formaliza como un nuevo requerimiento o si se retira en una futura versión.

Como equipo, este ejercicio evidenció la importancia de mantener alineados a los cuatro roles del proyecto (Analista, Diseñadora, Desarrollador y Tester) a lo largo de las distintas versiones del software, de manera que cualquier funcionalidad nueva que se agregue durante el desarrollo sea revisada y, en su caso, incorporada formalmente a los requerimientos antes de considerarse parte oficial del producto.

19. Referencias

Pressman, R. S. (2010). Ingeniería del software: un enfoque práctico (7a ed.). McGraw-Hill.

Beizer, B. (1990). Software Testing Techniques (2a ed.). Van Nostrand Reinhold.

Documento de análisis de requerimientos de la Calculadora, equipo del proyecto, asignatura Modelos de Prueba de Software, UPIICSA-IPN, secuencia 5CV50.

Documento de Diseño de Calculadora V1, equipo del proyecto, asignatura Modelos de Prueba de Software, UPIICSA-IPN, secuencia 5CV50.

Código fuente calculadora1.py (Calculadora V1) y calculadora2.py (Calculadora V2.0), Desarrollador del equipo del proyecto.

20. Anexos

Anexo A. Funciones identificadas en el código de la Calculadora V2.0

- `agregar_numero()`, `agregar_punto()`, `cambiar_signo()`, `calcular_porcentaje()`, `seleccionar_operacion()`, `calcular()`, `borrar_todo()`, `borrar_digito()`, `reutilizar_resultado()`, `limpiar_historial()`, `abrir_historial()`, `crear_boton_redondo()`.

Anexo B. Requerimientos originales (documento de análisis)

- RF-01 a RF-07 y RNF-01 a RNF-05, tal como se listan en las Tablas 3 y 4 del documento de análisis de requerimientos (ver secciones 3 y 15 de este documento).

Anexo C. Bocetos de referencia del documento de diseño

- Estado inicial, operación en proceso y resultado, división entre cero, intento de segundo punto decimal, historial de operaciones, reutilización de un resultado, diseño adaptable y retroalimentación visual (secciones 3.6 a 3.8 del documento "Diseño de Calculadora V1").

Nota final: este documento fue elaborado a partir de los materiales entregados por el equipo (análisis, diseño y código). Cualquier dato que no pudo confirmarse con dichos materiales —fechas, versiones de software, hardware utilizado, resultados de ejecución— se dejó marcado explícitamente como "Por confirmar", "Por definir" o "Pendiente de ejecución", en lugar de inventarse.